

TABLE OF CONTENT

Chương 1: Mở đầu	1
1.1. Đặt vấn đề	2
1.2. Lý do chọn đề tài	2
1.3. Mục tiêu đề tài	2
1.4. Phạm vi nghiên cứu	2
Chương 2: Các hệ thống liên quan	2
2.1. Gitleaks	2
2.2. Detect Secrets	2
2.3. Git Guardian	2
2.3. Truffle Hog	2
Chương 3: Cơ sở lý thuyết	3
3.1. Các kỹ thuật phát hiện dữ liệu nhạy cảm	3
3.2. An ninh chuỗi cung ứng phần mềm	3
3.3. Kiến trúc Visual Studio Code	3
3.4. Design Patterns áp dụng	3
Chương 4: Phân tích và thiết kế hệ thống	3
4.1. Phân tích yêu cầu	3
4.2. Kiến trúc tổng thể	3
4.3. Thiết kế chi tiết	3
Chương 4: Hiện thực và giải pháp kỹ thuật	4
4.1. Tích hợp Gitleaks Engine	4
4.2. Cơ chế Masking (Che dấu)	4
4.3. Giải pháp phân phối đa nền tảng	4
4.4. CI/CD Pipeline	4
Chương 5: Kiểm thử và Đánh giá	4
5.1. Kịch bản kiểm thử	4
5.2. So sánh hiệu năng	4
5.3. Độ chính xác	4
Chương 6: Kết luận và hướng phát triển mở rộng trong tương lai	4

Chương 1: Mở đầu

1.1. Đặt vấn đề

Nêu thực trạng lộ Key (dẫn chứng các vụ việc thực tế như: AWS, Firebase...). Nhấn mạnh sự nguy hiểm trong kỷ nguyên Cloud-native.

1.2. Lý do chọn đề tài

Các công cụ hiện tại (TruffleHog, Gitleaks CLI) mạnh nhưng UX kém, setup phức tạp. Developer lười chạy lệnh thủ công.

1.3. Mục tiêu đề tài

Xây dựng VS Code Extension tích hợp sâu, tự động hóa việc scan, che giấu (masking) và phát hiện thư viện độc hại.

1.4. Phạm vi nghiên cứu

VS Code API, Gitleaks engine, Supply Chain Attack in NPM.

Chương 2: Các hệ thống liên quan

2.1. Gitleaks

2.1.1. Giới thiệu

2.1.2. Ưu điểm

2.1.3. Nhược điểm

2.2. Detect Secrets

2.2.1. Giới thiệu

2.2.2. Ưu điểm

2.2.3. Nhược điểm

2.3. Git Guardian

2.3.1. Giới thiệu

2.3.2. Ưu điểm

2.3.3. Nhược điểm

2.3. Truffle Hog

2.3.1. Giới thiệu

2.3.2. Ưu điểm

2.3.3. Nhược điểm

Chương 3: Cơ sở lý thuyết

3.1. Các kỹ thuật phát hiện dữ liệu nhạy cảm

- Cơ chế entropy (độ hỗn loạn thông tin), Regular Expression trong phát hiện pattern.

3.2. An ninh chuỗi cung ứng phần mềm

- Typosquatting, Malware injection trong NPM.

3.3. Kiến trúc Visual Studio Code:

- Giải thích cơ chế hoạt động: Extension Host process vs. UI process.
- Language Server Protocol (LSP) - nếu có ý định dùng để highlight lỗi trực tiếp trên code.

3.4. Design Patterns áp dụng

- *Ví dụ:* **Decorator Pattern** cho tính năng Masking (bao bọc text gốc bằng text hiển thị), **Adapter Pattern** để giao tiếp với Gitleaks engine.

Chương 4: Phân tích và thiết kế hệ thống

4.1. Phân tích yêu cầu

- **Functionals**
- **Non-functional Requirements**
 - *Hiệu năng:* Scan phải nhanh (dưới bao nhiêu ms?), không làm đơ IDE.
 - *Bảo mật:* Extension không được gửi code của người dùng ra ngoài (Privacy First).

4.2. Kiến trúc tổng thể

- Vẽ sơ đồ khối mô tả cách Extension tương tác với Gitleaks Binary, Database Malware (Alkido), và VS Code Editor API.

4.3. Thiết kế chi tiết

- Flowchart quy trình Scan khi người dùng bấm Save hoặc Commit.
- Sequence Diagram cho tính năng "Masking Secret".

Chương 5: Hiện thực và giải pháp kỹ thuật

4.1. Tích hợp Gitleaks Engine

- Cách gọi bất đồng bộ, xử lý stdout/stderr.

4.2. Cơ chế Masking

- Sử dụng VSCode API như thế nào?

4.3. Giải pháp phân phối đa nền tảng

- Đây là điểm sáng kỹ thuật (Key Highlight) cho vấn đề ở trên.

4.4. CI/CD Pipeline

- Quy trình build và release tự động.

Chương 6: Kiểm thử và Đánh giá

5.1. Kịch bản kiểm thử

- Test với repo 10 file, 1000 file, repo có history dài.

5.2. So sánh hiệu năng

- Tốc độ scan so với chạy CLI thuần túy.

5.3. Độ chính xác

- Tỷ lệ False Positive (báo động giả) là bao nhiêu?

Chương 7: Kết luận và hướng phát triển mở rộng trong tương lai